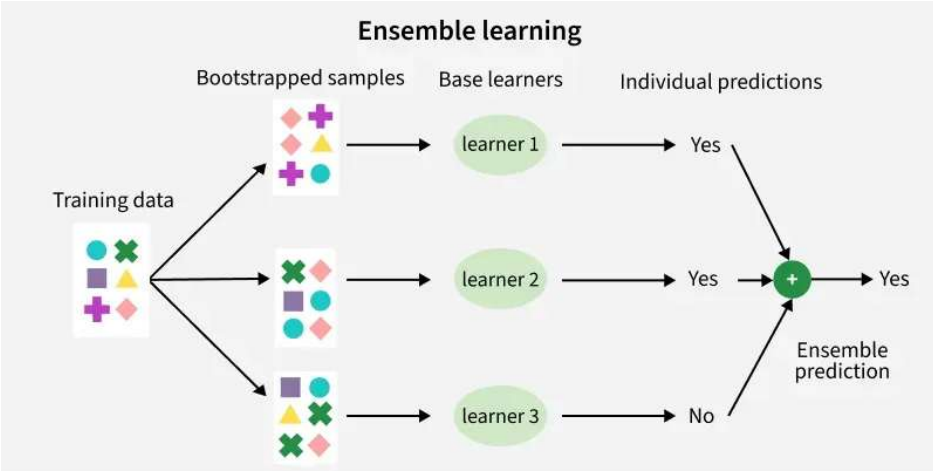
XGBoost , Ensemble & Advanced ML

- Varun Raste

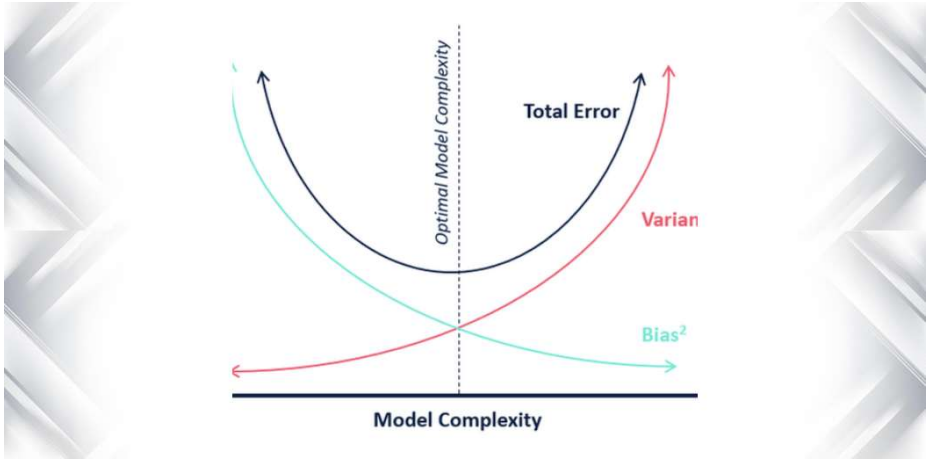
What is Ensemble Learning ?

- Technique of combining multiple models to boost prediction accuracy
- Helps overcome limitations of individual weak learners
- Reduces variance, improves stability, and enhances generalization
- Works through methods like Bagging, Boosting, and Stacking
- Backbone for many high-performing ML solutions today



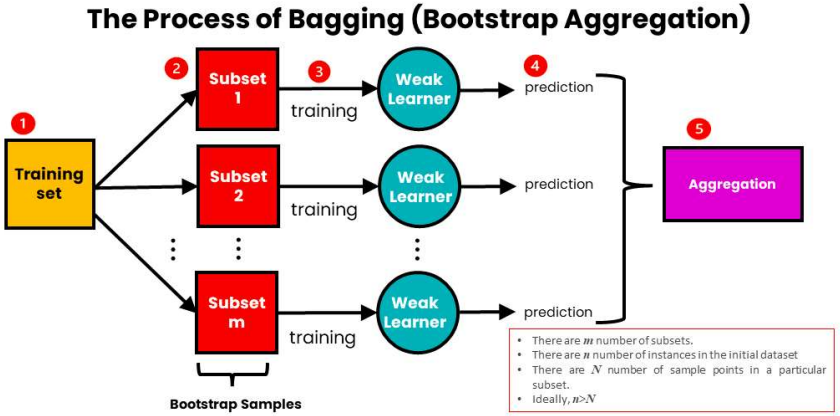
Variance-Bias Tradeoff

- Bias: Error from overly simplistic model assumptions → underfitting
- Variance: Error from model sensitivity to training data → overfitting
- Goal is to find the optimal balance for best generalization
- Ensemble methods help reduce variance while keeping bias low
- Proper tuning (depth, learning rate, regularization) achieves stability



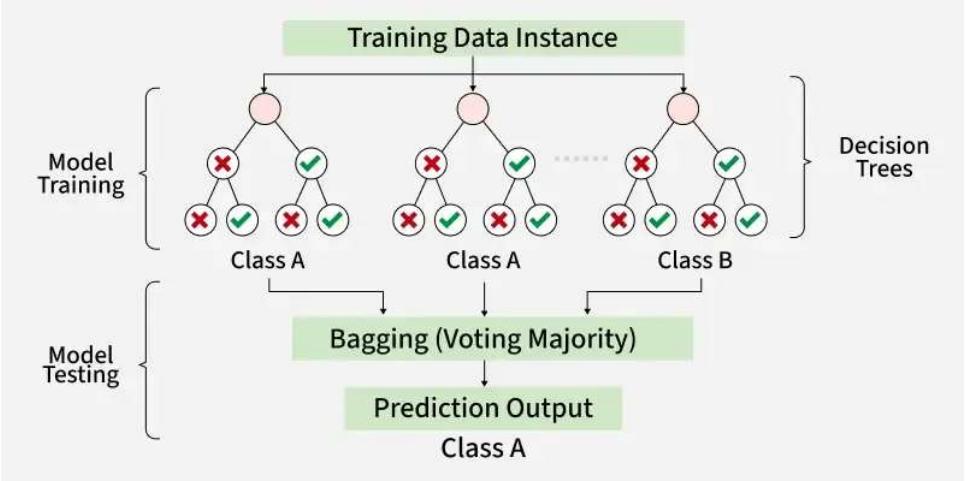
Bagging (Bootstrap Aggregation)

- Trains multiple models on different bootstrapped samples
- Reduces variance and improves model stability
- Models run independently → highly parallelizable
- Final prediction via voting (classification) or averaging (regression)
- Foundation technique behind Random Forest



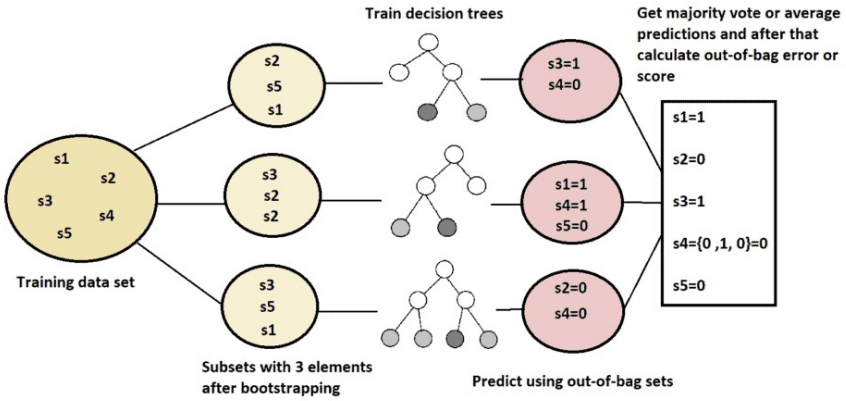
Random Forest – Decision Tree”s” version 2.0

- Builds an ensemble of decision trees, each trained on a different random slice of the dataset, creating a powerful “forest” of diverse learners.
- Introduces randomness at both data and feature level, ensuring trees are decorrelated and less prone to overfitting.
- Excels on complex, high-dimensional, and noisy tabular datasets where single trees may struggle.
- Naturally handles missing values and nonlinear relationships without heavy preprocessing.
- Delivers strong, reliable performance with minimal tuning, making it a go-to baseline in many ML pipelines.

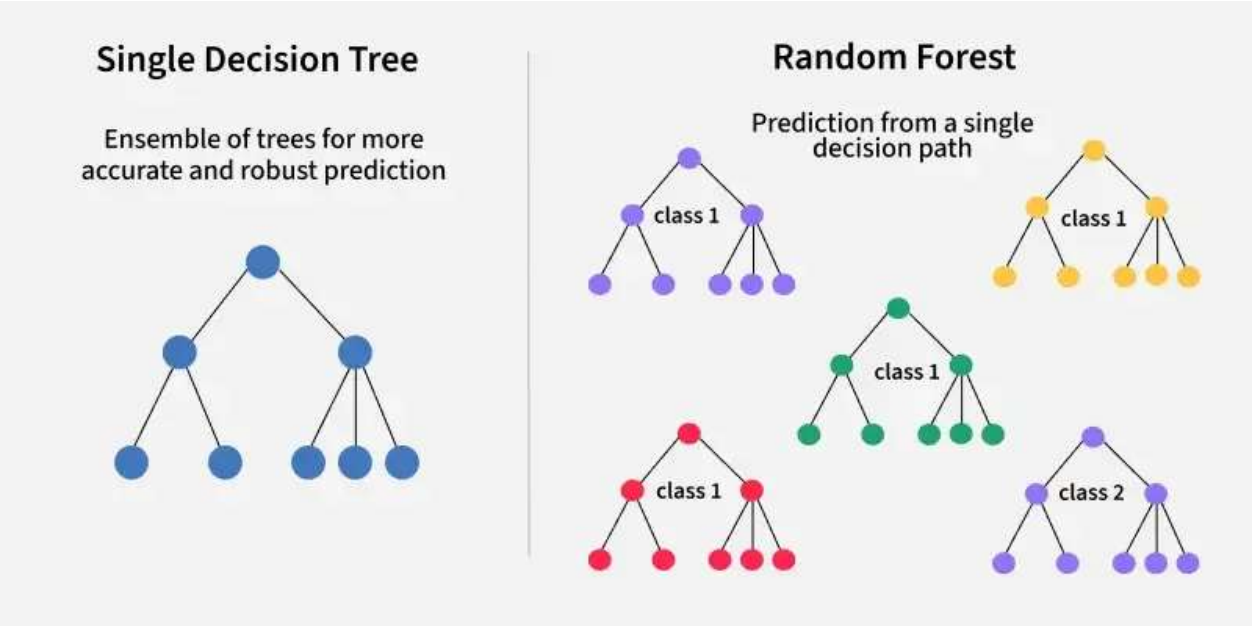


Random Forest – Decision Tree”s” version 2.0

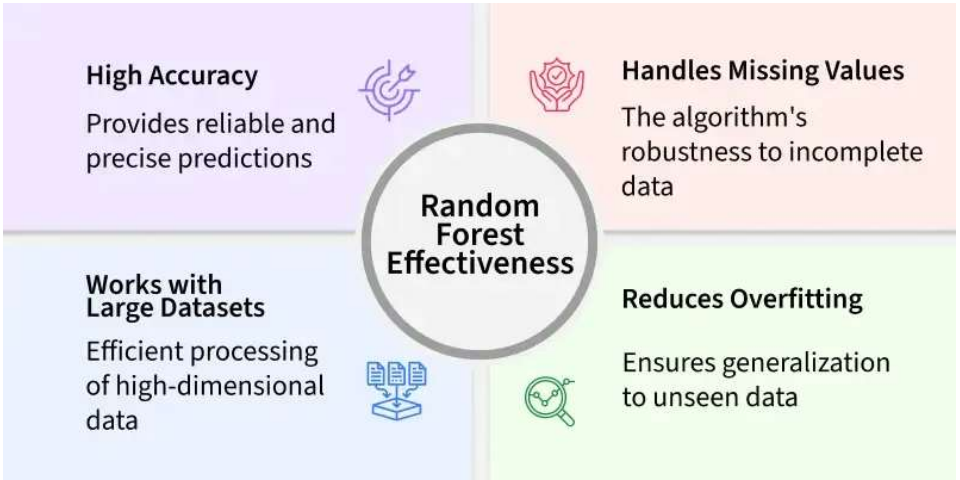
- Trains many decision trees on bootstrapped samples, ensuring each tree sees a slightly different version of the dataset.
- At every split, each tree considers only a random subset of features, reducing correlation and improving diversity across trees.
- Final prediction is produced through majority voting (classification) or averaging (regression), smoothing out individual tree noise.
- This layered randomness drastically reduces variance, making the model stable even with messy or noisy data.
- Includes Out-of-Bag (OOB) error as an internal validation mechanism, removing the need for a separate validation split.



Note : Every DT should get same number of samples as original data size *

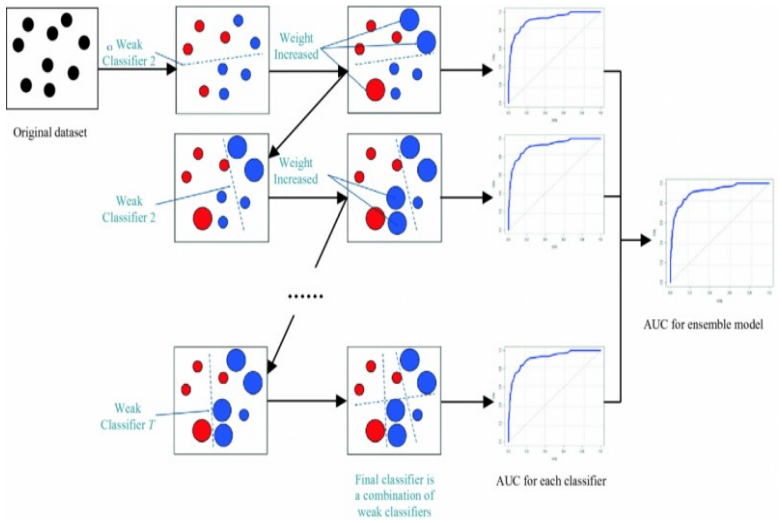


Descriptions for DT & RF mentioned in above image are interchanged , make a note of it*



Gradient Boosting

- Builds models sequentially, where each new tree focuses on fixing the mistakes (residuals) made by the previous trees.
- Starts with a simple base model and gradually transforms it into a powerful predictor through iterative refinement.
- Unlike bagging, boosting reduces bias, making the final model capable of capturing complex patterns.
- Highly flexible — supports different loss functions (MSE, LogLoss, MAE) depending on the problem.
- Forms the foundation for advanced algorithms like XGBoost, LightGBM, and CatBoost, widely used in real-world ML systems.

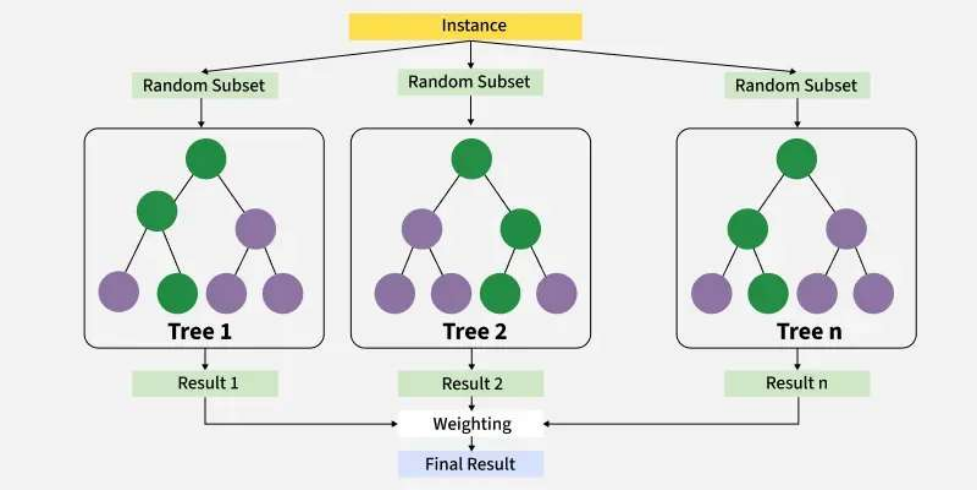


Gradient Boosting - Working

- Begins with a simple model (often a small tree), then fits new trees on the residual errors left by previous predictions.
- Each tree learns to correct specific weaknesses, allowing the ensemble to improve stage by stage.
- A learning rate controls how much each new tree contributes, helping prevent overfitting while stabilizing learning.
- Uses additive modeling — the final prediction is the sum of outputs from many small, focused trees.
- Supports customizable loss functions, giving it flexibility across regression, classification, and ranking tasks.

XGBoost

- “Extreme Gradient Boosting,” a highly optimized upgrade of traditional GBM designed for speed and performance.
- Adds regularization (L1 & L2) to control overfitting, making it more stable on noisy and high-dimensional data.
- Uses parallelized tree construction, efficient caching, and hardware-level optimizations for lightning-fast training.
- Handles missing values intelligently by learning optimal default split directions.
- A top performer in Kaggle competitions and industry ML pipelines due to its balance of accuracy, speed, and flexibility.
- .

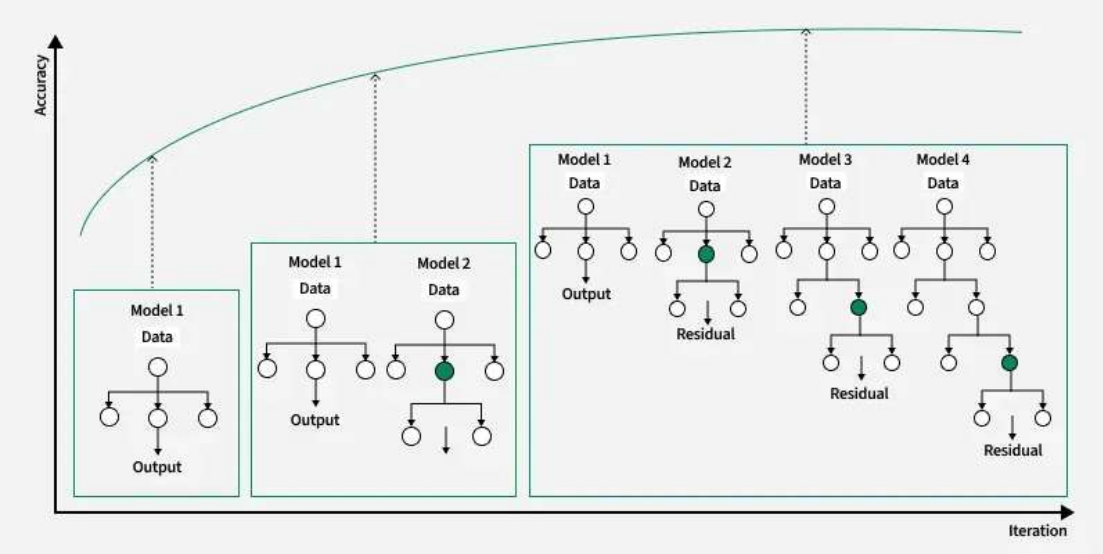


XGBoost : Key Features

- Uses a regularized objective function that balances model complexity with accuracy, preventing overfitting even on noisy datasets.
- Employs a weighted quantile sketch algorithm to efficiently find optimal split points on large, sparse feature spaces.
- Provides built-in cross-validation, early stopping, and advanced tree boosters (tree & linear) for flexible experimentation.
- Automatically learns how to handle missing values, reducing preprocessing effort and improving robustness.
- Highly tunable with parameters like depth, subsampling, and learning rate — enabling fine-grained control over model behavior..

LightGBM

- A fast, high-performance gradient boosting framework developed by Microsoft, optimized for large-scale machine learning.
- Uses a leaf-wise tree growth strategy (best-first), enabling deeper, more informative splits compared to traditional level-wise methods.
- Extremely efficient on massive datasets with high dimensionality, thanks to its histogram-based approach.
- Consumes significantly less memory than many other boosting algorithms while maintaining top-tier accuracy.
- Widely adopted in production systems where speed, scalability, and performance are critical..

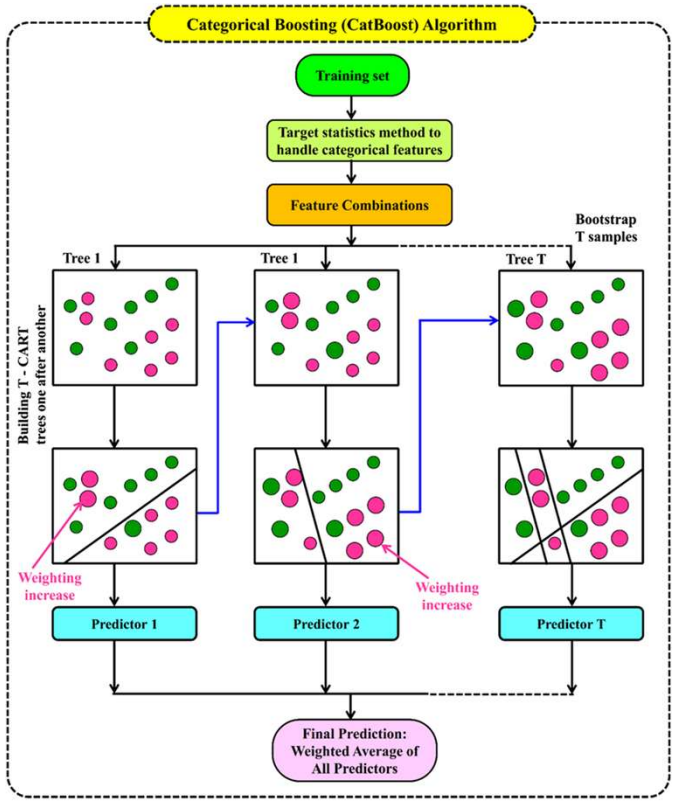


LightGBM : Key Features

- Uses histogram-based splitting, grouping continuous features into bins, drastically speeding up training and reducing memory usage.
- Grows trees leaf-wise, focusing on the split that maximizes loss reduction, often outperforming level-wise methods in accuracy.
- Provides native support for handling categorical features using optimized decision rules.
- Highly efficient on large, sparse datasets and offers seamless GPU acceleration, making it ideal for production-scale ML.
- Can be prone to overfitting due to aggressive leaf-wise growth, requiring careful tuning of depth and regularization.

CatBoost

- A powerful gradient boosting algorithm by Yandex, designed to handle categorical features natively without one-hot or label encoding.
- Uses ordered boosting, a unique technique that avoids target leakage and produces more stable, unbiased models.
- Performs exceptionally well on noisy, small, or mixed-type datasets where other models may struggle.
- Reduces preprocessing effort significantly, making it easy to plug into real-world ML workflows.
- Known for delivering strong accuracy “out-of-the-box,” often requiring minimal hyperparameter tuning.



CatBoost : Key Features

- Native Categorical Handling: Converts categorical columns into powerful target statistics using ordered encoding, eliminating the need for manual one-hot or label encoding.
- Ordered Boosting: Prevents prediction shift and target leakage by ensuring each data point is trained only on prior information.
- Symmetric Trees: Builds balanced, depth-wise symmetric trees that make both training and inference faster and more hardware-efficient.
- Strong Out-of-the-Box Performance: Delivers high accuracy with minimal hyperparameter tuning, making it ideal for fast experimentation.
- Robust to Noisy and Non-Stationary Data: Handles real-world messy datasets better than many traditional boosting methods.

When to use which model ?

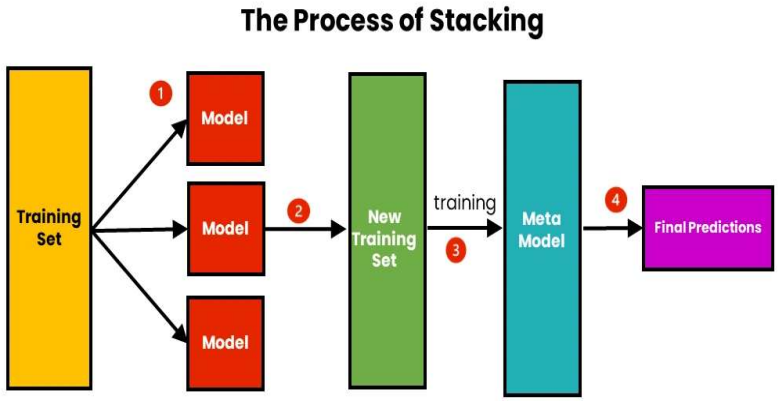
Feature / Criterion	Random Forest	GBM	XGBoost	LightGBM	CatBoost
Training Speed	Moderate	Slow	Fast (optimized GBM)	**Very Fast** (best)	Moderate
Inference Speed	Moderate	Slower	Fast	**Very Fast**	Fast
Overfitting Control	Good (bagging reduces variance)	Requires tuning	Strong (L1/L2 regularization)	Needs depth constraints	Very good (ordered boosting)
Handling Categorical Features	Needs encoding	Needs encoding	Needs encoding	Limited support	**Best (native handling)**
Handling Large Datasets	Good but slower	Not ideal	Good	**Excellent**	Good
Handling Sparse Data	Moderate	Moderate	**Excellent**	**Excellent**	Good
Hyperparameter Tuning	Low effort	High effort	High effort	High effort	**Low effort**
Performance on Noisy Data	Strong	Moderate	Strong	Moderate	**Strong**
Memory Usage	High	Moderate	Moderate	**Low**	Moderate
Accuracy on Complex Patterns	Good	Good	**High**	**High**	**High**
Interpretability	Moderate	Better than others	Low	Low	Low
Best Use Cases	Baseline models, noisy data	Bias reduction, customizable losses	Structured ML competitions, sparse data	Huge datasets, production ML systems	Categorical-heavy datasets, minimal preprocessing

Model Parameters

Parameter	Meaning / What It Does	RF	GBM	XGBoost	LightGBM	CatBoost
n_estimators	Number of trees; more trees = better but slower	100–500	100–500	200–1000	200–1000	200–1000
max_depth	Maximum depth of each tree; controls model complexity	5–20; reduce overfitting	3–6	3–10	3–12	6–10
learning_rate	Shrinks each tree's contribution; lower = more stable	—	0.01–0.1	0.01–0.1	0.01–0.1	0.03–0.3
subsample	% of rows used per tree; prevents overfitting	—	0.7–1.0	0.6–0.9	`bagging_fraction` (0.7–1.0)	0.7–1.0
colsample_bytree / feature_fraction	% of features used per tree; adds randomness	`max_features` (0.3–1.0)	—	0.5–1.0	0.5–1.0	`rsm` (0.8–1.0)
min_samples_split / min_child_weight / min_data_in_leaf	Minimum samples needed to create a split; larger = less overfitting	RF: 2–10	GBM: 2–20	XGB: 1–10	LGBM: 20–50	CB: 1–20
regularization (L1/L2)	Adds penalties to control overfitting	Not used	Mild (depends on lib)	L1=alpha, L2=lambda (0–10)	lambda_l1/l2 (0–10)	`l2_leaf_reg` (1–10)
gamma / min_gain_to_split	Minimum loss reduction needed to split a node	—	—	gamma = 0–5	min_gain_to_split = 0–5	—
max_leaf_nodes / num_leaves	Controls the number of terminal nodes	RF: optional	10–50	—	31–255	Auto
objective / loss_function	Defines the loss used for training	Gini/Entropy	MSE, MAE, Logloss	Regression, binary, ranking	All major losses	All major losses
categorical handling	How model processes categorical features	Needs encoding	Needs encoding	Needs encoding	Basic handling	**Best in class – ordered encoding**
early_stopping_rounds	Stops training when improvement stalls	Not common	n_iter_no_change=5–20	10–50	20–50	`od_wait` = 20–50

Stacking

- Combines predictions from multiple models to make a stronger final model.
- Base models are trained independently on the same data.
- Their out-of-fold predictions are used as inputs to a meta-model.
- The meta-model learns how to best blend the base models.
- Often improves accuracy by using strengths of different algorithms.



Resources :

Github repo for Code : https://github.com/varunchach/Ensemble_Advanced_ML.git

General Resources for Reading about ML Algos:

XGBoost : 1. https://xgboost.readthedocs.io/en/stable/
Catboost : https://catboost.ai/
LightGBM: https://lightgbm.readthedocs.io/en/stable/
Sklearn : https://scikit-learn.org/ https://scikit-learn.org/stable/modules/ensemble.html Scikit-learn

YT Playlist for boosting algos :

1. https://youtube.com/playlist?list=PLHfe9-lMyRPYg1zDI1iQkwUzLbjPlnUdY&si=c3twweW42wf8vcfB
2. https://youtube.com/playlist?list=PLmPJQXJiMoUWfMvIqAn0VI_LHlw4AmVrb&si=4UwPseo81Sq7oBIa
3. https://youtube.com/playlist?list=PLTDARY42LDV7WGmlzZtY-w9pemyPrKNUZ&si=O8LJWBpW0yTXkTU5